

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 1
з дисципліни „Професійний Python”
на тему „Організація професійного Python-проєкту та Git.”

Виконала:
студентка 3 курсу
групи Феп-32
Клімашук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Номер і тема лабораторної роботи

Лабораторна робота №1. Організація професійного Python-проєкту та Git.

2. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

3. Номер індивідуального варіанта

Варіант №9.

4. Постановка задачі

Створити консольний Python-застосунок «Система обліку книг у бібліотеці».

- **Модель даних:** Для книги зберігати назву, автора, рік видання, унікальний код (ID/ISBN) та статус (доступна / видана).
- **Функціональні вимоги:**
 - Додавання нової книги до системи.
 - Пошук книги за автором або назвою.
 - Видача книги читачеві (зміна статусу на «видана»).
 - Повернення книги до бібліотеки (зміна статусу на «доступна»).
 - Отримання списку всіх виданих на даний момент книг.
- **Архітектурні вимоги:** Використання src-layout, Type Hints, кастомні винятки, CLI-меню, конфігурація `pyproject.toml`, `.gitignore`, `README.md` та ведення історії змін через Git.

5. Структура проєкту

Проєкт побудовано за стандартом src-layout:

- **library_management/** — кореневий каталог проєкту.
 - **.gitignore** — файл із переліком тимчасових файлів та каталогів, що ігноруються системою Git.
 - **pyproject.toml** — конфігураційний файл метаданих та залежностей проєкту.
 - **README.md** — файл із загальним описом проєкту та інструкціями щодо розгортання й запуску.
 - **report/** — каталог для зберігання звіту з лабораторної роботи у форматі PDF чи DOCX.
 - **src/** — каталог джерельного коду (згідно зі стандартом src-layout).
 - **library_management/** — основний пакет програми.

- **__init__.py** — службовий файл для позначення директорії як Python-пакета.
- **exceptions.py** — модуль із кастомними класами винятків.
- **main.py** — точка входу в застосунок та реалізація консольного інтерфейсу.
- **models.py** — модуль із моделями даних (клас Book).
- **services.py** — модуль із функціями бізнес-логіки.

6. Опис створених modules

__init__.py: Маркує директорію як Python-пакет та визначає версію пакета (`__version__ = "0.1.0"`).

exceptions.py: Містить ієрархію кастомних винятків (`LibraryError`, `BookNotFoundError`, `BookAlreadyIssuedError`, `BookNotIssuedError`) для обробки помилок бізнес-логіки.

models.py: Описує структуру даних книги `Book` з використанням `@dataclass`, а також містить розраховувану властивість `status_display`.

services.py: Модуль бізнес-логіки. Містить функції для додавання книг, пошуку за назвою/автором, видачі, повернення та отримання списку виданих книг.

main.py: Точка входу в застосунок. Містить інтерактивне консольне меню, функцію валідації введення чисел з клавіатури та відображення списку книг у табличному вигляді.

7. Текст основних фрагментов коду

Файл `src/library_management/models.py`

Python

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Book:
```

```
    book_id: int
```

```
    title: str
```

```
    author: str
```

```
    year: int
```

```
    is_issued: bool = False
```

```
@property
```

```
def status_display(self) -> str:
```

```
    return "Видана" if self.is_issued else "Доступна"
```

Файл `src/library_management/exceptions.py`

Python

```
class LibraryError(Exception):
    """Базовий виняток для системи бібліотеки."""

class BookNotFoundError(LibraryError):
    """Викликається, коли книгу не знайдено."""

class BookAlreadyIssuedError(LibraryError):
    """Викликається при спробі видати вже видану книгу."""

class BookNotIssuedError(LibraryError):
    """Викликається при спробі повернути книгу, яка не була видана."""
```

Файл `src/library_management/services.py`

Python

```
from library_management.exceptions import (
    BookAlreadyIssuedError,
    BookNotIssuedError,
    BookNotFoundError,
)
from library_management.models import Book

def add_book(books: list[Book], title: str, author: str, year: int) -> Book:
    """Додає нову книгу до бібліотеки."""
    new_id = max([b.book_id for b in books], default=0) + 1
    new_book = Book(book_id=new_id, title=title, author=author, year=year)
    books.append(new_book)
    return new_book

def find_books(books: list[Book], query: str) -> list[Book]:
    """Шукає книги за назвою або автором (без урахування регістру)."""
    q = query.lower().strip()
    return [
        b for b in books
        if q in b.title.lower() or q in b.author.lower()
    ]
```

```
]
```

```
def find_book_by_id(books: list[Book], book_id: int) -> Book | None:
```

```
    """Шукає книгу за її ID."""
```

```
    for book in books:
```

```
        if book.book_id == book_id:
```

```
            return book
```

```
    return None
```

```
def issue_book(books: list[Book], book_id: int) -> Book:
```

```
    """Змінює статус книги на 'видана'."""
```

```
    book = find_book_by_id(books, book_id)
```

```
    if book is None:
```

```
        raise BookNotFoundError(f"Книгу з ID {book_id} не знайдено.")
```

```
    if book.is_issued:
```

```
        raise BookAlreadyIssuedError(f"Книга '{book.title}' вже видана.")
```

```
    book.is_issued = True
```

```
    return book
```

```
def return_book(books: list[Book], book_id: int) -> Book:
```

```
    """Повертає книгу до бібліотеки."""
```

```
    book = find_book_by_id(books, book_id)
```

```
    if book is None:
```

```
        raise BookNotFoundError(f"Книгу з ID {book_id} не знайдено.")
```

```
    if not book.is_issued:
```

```
        raise BookNotIssuedError(f"Книга '{book.title}' не була видана.")
```

```
    book.is_issued = False
```

```
    return book
```

```
def get_issued_books(books: list[Book]) -> list[Book]:
```

```
    """Повертає список усіх виданих книг."""
```

```
    return [b for b in books if b.is_issued]
```

8. Пояснення pyproject.toml

Файл pyproject.toml відповідає стандарту PEP 518/PEP 621 для конфігурації проєктів Python:

[build-system]: Визначає інструмент збірки setuptools та його бекенд.

[project]: Вказує назву пакета (library-management), версію (0.1.0), опис та сумісну версію Python (≥ 3.11).

[project.scripts]: Реєструє консольну команду library-management, яка запускає точку входу library_management.main:main при інсталяції через `pip install -e ..`

Вміст файлу:

Ini, TOML

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "library-management"
version = "0.1.0"
description = "Library Book Management System"
requires-python = ">=3.11"

[project.scripts]
library-management = "library_management.main:main"
```

9. Вміст .gitignore

Plaintext

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

10. Фрагмент README.md

Файл надає стислий опис проекту та поетапні інструкції для розгортання й запуску:

```

PS C:\Users\Veronika\Phyton\library_management> python -m venv .venv
PS C:\Users\Veronika\Phyton\library_management> .venv\Scripts\activate
(.venv) PS C:\Users\Veronika\Phyton\library_management> pip install -e .
Obtaining file:///C:/Users/Veronika/Phyton/library_management
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: library-management
Building editable for library-management (pyproject.toml) ... done
Created wheel for library-management: filename=library_management-0.1.0-0.editable-py3-n
1652 sha256=de2e89ba82490cdea0a7c05831dda22796d8da4cee05a217605614debd125deb
Stored in directory: C:\Users\Veronika\AppData\Local\Temp\pip-ephem-wheel-cache-wgz2sovk
4e721245318e2805e86c7e347aab011652179c6814efa06405
Successfully built library-management
Installing collected packages: library-management
Successfully installed library-management-0.1.0

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Veronika\Phyton\library_management> python -m library_management.main

```

```

(.venv) PS C:\Users\Veronika\Phyton\library_management> python -m library_management.main

--- СИСТЕМА ОБЛІКУ КНИГ У БІБЛІОТЕЦІ ---
1. Показати всі книги
2. Додати книгу
3. Знайти книгу (за назвою або автором)
4. Видати книгу
5. Повернути книгу
6. Показати список виданих книг
0. Вихід
Оберіть опцію: 4
Введіть ID книги для видачі: 1

Успіх: Книгу 'Кобзар' (ID 1) успішно видано читачеві!

```

Перегляд фінансової статистики:

```

--- Фінансова статистика ---
Поточна виручка (заброньовано): 8400.00 грн
Максимально можлива виручка: 12100.00 грн

--- СИСТЕМА БРОНЮВАННЯ ГОТЕЛЮ ---
1. Показати всі номери
2. Показати вільні номери
3. Забронювати номер
4. Скасувати бронювання
5. Знайти найдешевший вільний номер
6. Показати фінансову статистику
0. Вихід
Оберіть опцію: 

```

12. Результат git status

```

(.venv) PS C:\Users\Veronika\Phyton\library_management> git status
On branch main
nothing to commit, working tree clean
(.venv) PS C:\Users\Veronika\Phyton\library_management> 

```

13. Результат git log --oneline

```
(.venv) PS C:\Users\Veronika\Phyton\library_management> git log --oneline
67e62f8 (HEAD -> main) Initial commit: Library Management System (Variant 9)
(.venv) PS C:\Users\Veronika\Phyton\library_management>
```

14. Відповіді на контрольні запитання

1. Що таке Python-проект? — Це сукупність коду, модулів, залежностей, конфігураційних файлів та документації, організованих за єдиною структурою для вирішення конкретного завдання.
2. Чим проект відрізняється від одного Python script? — Скрипт — це один файл .py. Проект має чітку модульну структуру, ізольоване середовище, конфігурацію пакетування, документацію та контроль версій.
3. Що таке Python module? — Це окремий .py файл, який містить код (функції, класи), призначений для повторного використання.
4. Що таке Python package? — Директорія, яка містить модулі та файл __init__.py.
5. Для чого потрібний __init__.py? — Позначає директорію як Python package і дозволяє здійснювати імпорт з неї.
6. Що таке src-layout? — Структура проекту, де сирцевий код розробки розміщується в окремій директорії src/.
7. Які переваги має src-layout? — Запобігає випадковому імпорту незаінстальованого коду під час тестування та чітко відокремлює код програми від додаткових службових файлів.
8. Для чого використовується virtual environment? — Для ізоляції залежностей та бібліотек одного проекту від інших проектів або системного Python.
9. Як створити .venv? — За допомогою команди `python -m venv .venv`.
10. Як активувати virtual environment? — у Windows: `.venv\Scripts\activate`; у Linux/macOS: `source .venv/bin/activate`.
11. Для чого використовується pip? — Для встановлення та управління сторонніми пакетами й бібліотеками Python.
12. Чому рекомендується `python -m pip`? — Гарантує, що pip запускається саме з того інтерпретатора Python, який активний у даному віртуальному середовищі.
13. Для чого потрібний `pyproject.toml`? — Є єдиним стандартом конфігурації проектів у Python (метадані, збірка, скрипти).
14. Що містить секція `[project]`? — Назву проекту, версію, опис, вимоги до версії Python, залежності та точки запуску скриптів.
15. Що таке `editable installation`? — Встановлення пакета через `pip install -e .`, що дозволяє відразу тестувати зміни у коді без повторної інсталяції.
16. Для чого використовується `if __name__ == "__main__":`? — Для того, щоб блок коду запускався тільки при безпосередньому виконанні файла як головного скрипта, а не при імпорті.
17. Що таке Git? — Розподілена система контролю версій.
18. Що таке repository? — База даних Git, яка зберігає всі файли проекту та повну історію їхніх змін.

19. Що таке commit? — Фіксований збережений стан файлів проєкту в історії Git.
20. Для чого використовується git status? — Показує поточний стан робочого дерева (змінені, невідстежувані або підготовлені файли).
21. Для чого git add? — Додає зміни до області підготовки (staging area) перед створенням комміту.
22. Для чого git commit? — Фіксує підготовлені зміни в історії версій із пояснювальним повідомленням.
23. Що показує git log? — Історію виконаних коммітів у репозиторії.
24. Для чого потрібен .gitignore? — Вказує файли й директорії, які Git не повинен відстежувати (кеш, віртуальні середовища, секрети).
25. Чому .venv не потрібно комітити? — Оскільки воно містить системно-залежні бінарні файли великого розміру, а всі необхідні залежності описуються у конфігурації проєкту.
26. Для чого потрібний README? — Надає опис проєкту, інструкцію зі встановлення, запуску та інформацію про авторів.
27. Що таке type hints? — Вказівка типів даних у коді для покращення його читабельності та автоматичної перевірки аналізаторами.
28. Для чого програму розділяють на modules? — Для забезпечення структурованості, легкості читання, спрощення тестування та повторного використання коду.
29. Що таке business logic? — Основна логіка обробки даних, розрахунків та правил роботи програми, відокремлена від інтерфейсу користувача.
30. Чому потрібно відокремлювати application entry point від business logic? — Це дозволяє змінювати або замінювати інтерфейс (наприклад, з консольного на веб або GUI) без зміни основної логіки програми.

15. Висновок

У лабораторній роботі здійснено перехід від створення окремих Python scripts до організації повноцінного програмного проєкту. Використання src-layout, модульної архітектури, virtual environment, pyproject.toml, Git та README.md сформувало фундамент для подальших лабораторних робіт.

Проєкт, створений у лабораторній роботі №1, доцільно використовувати як основу наступних робіт курсу, куди поступово додаватимуться складні структури даних, ООП, тестування, збереження даних та робота з API.